

Curso: Señales Biomédicas | Institución: UPCH / PUCP | Herramientas: Python (NumPy, Matplotlib, WFDB, SciPy)

1. Fundamentos de la Sesión de Laboratorio: Conceptos Básicos

¿Qué es una Señal Biomédica?

Es toda señal de origen biológico que transporta información sobre los procesos fisiológicos del cuerpo humano, desde nivel molecular o celular hasta sistemas orgánicos complejos.

- **ECG:** Electrocardiograma (actividad eléctrica del corazón).
- **EMG:** Electromiograma (actividad muscular).
- **EEG:** Electroencefalograma (actividad cortical cerebral).

Muestreo y Tiempo Discreto

Las señales analógicas del cuerpo son continuas. Para procesarlas en computadoras (Jupyter/Colab), se digitalizan mediante un proceso de muestreo:

- **Frecuencia de Muestreo (f_s):** Número de muestras por segundo (Hz).
- **Periodo de Muestreo (T_s):** $T_s = 1 / f_s$.
- **Eje de Tiempo Discreto:** $t[n] = n \cdot T_s$, donde n es el índice entero de la muestra.

2. Análisis de Señales Reales en PhysioNet (WFDB)

Estructura de Datos y Extracción Clínica

Para trabajar con registros clínicos reales (como la base de datos de arritmias de MIT-BIH), el laboratorio utiliza la librería `wfdb`, requiriendo identificar unívocamente la base de datos y el registro:

- **Identificadores:** `DATABASE = "mitdb"` y `RECORD = "100"` (o "20").
- **Carga del Registro:** `record = wfdb.rdrecord(RECORD, pn_dir=DATABASE)`
- **Frecuencia de Muestreo del Registro:** `record.fs` (ej. 360 Hz para mitdb).
- **Dimensiones y Canales (`record.p_signal`):** La matriz de señales físicas almacena los canales en las columnas. Para aislar un canal específico (por ejemplo, derivación MLII) se extrae mediante `record.p_signal[:, CHANNEL]`, donde `CHANNEL` es un índice entero (0, 1, etc.).

3. Generación y Visualización de Señales Sintéticas (Código del Laboratorio)

Implementación 1: Señal de 120 Hz

```
f1 = 120
w1 = 2 * np.pi * f1
A = 1
phi = 0
fs = 1e3
ts = 1 / fs
n = np.arange(1000)
t = n * ts

x1 = A * np.sin(w1 * t + phi)
plt.plot(x1)
plt.xlim(0, 50)
plt.show()
```

Genera una senoide de 120 Hz a 1 kHz de muestreo. `plt.xlim(0, 50)` restringe la vista a las primeras 50 muestras para inspeccionar detalle de onda.

Implementación 2: Señal de 20 Hz

```
f2 = 20
w2 = 2 * np.pi * f2
A = 1
phi = 0
fs = 1e3
ts = 1 / fs
n = np.arange(1000)
t = n * ts

x2 = A * np.sin(w2 * t + phi)
plt.plot(x2)
plt.xlim(0, 50)
plt.show()
```

Genera una senoide de menor frecuencia (20 Hz). Al tener menor frecuencia, se aprecia un periodo más largo por cada 50 muestras graficadas.

4. Análisis en el Dominio de la Frecuencia: Espectro de Magnitud

Uso de `plt.magnitude_spectrum`

```
plt.magnitude_spectrum(
    x3, scale="dB", Fs=fs
)
plt.show()
```

Esta función calcula y grafica el espectro de magnitud de una señal discreta (ej. `x3`). El parámetro `scale="dB"` convierte la magnitud a decibeles ($20 \log_{10}(|X|)$), facilitando la visualización de componentes armónicos débiles y picos de ruido.

Parámetros de Espectro

- **`x3`**: Vector de la señal analizada.
- **`scale="dB"`**: Escala logarítmica de potencia/amplitud.
- **`Fs=fs`**: Frecuencia de muestreo empleada para calibrar correctamente el eje X de frecuencias (Hz).

5. Filtrado Digital en Procesamiento de Señales Biomédicas

Necesidad y Conceptos Clave

Las señales biomédicas capturadas en la práctica clínica suelen estar contaminadas con ruido instrumental, artefactos de movimiento o interferencia de la red eléctrica (50/60 Hz). Los filtros digitales permiten aislar las bandas de frecuencia de interés clínico sin alterar la morfología de la señal.

Tipos de Filtros Principales

- **Filtros FIR (Finite Impulse Response):**
 - Respuesta impulsional finita.
 - No tienen polos (solo ceros en el plano Z), lo que garantiza **estabilidad incondicional**.
 - Permiten lograr **fase lineal** exacta, evitando distorsión temporal en ondas complejas (como el complejo QRS del ECG).
- **Filtros IIR (Infinite Impulse Response):**
 - Respuesta impulsional infinita debido al uso de retroalimentación (polos y ceros).
 - Altamente eficientes computacionalmente (requieren menor orden de filtro para cortes abruptos).
 - Ejemplo clásico: Filtros Butterworth o Chebyshev para pasabanda de ECG (0.5 Hz - 40 Hz).

Función de Transferencia (H(z))

Matemáticamente, un filtro digital LTI se define en el dominio Z a través de su función de transferencia, que relaciona la salida **Y(z)** con la entrada **X(z)**:

$$H(z) = \frac{Y(z)}{X(z)} = \frac{\sum_{k=0}^M b_k z^{-k}}{1 + \sum_{j=1}^N a_j z^{-j}}$$

Donde los coeficientes **b_k** corresponden a los ceros (feedforward) y **a_j** a los polos (feedback). En Python, estos coeficientes se obtienen mediante funciones de diseño de `scipy.signal` y se aplican mediante `scipy.signal.lfilter()` o `filtfilt()`.